

ChefBot

Chatbot de Receitas com Arquitetura Híbrida

NLP Clássico + Large Language Model | Python · Flask · NLTK · Transformers

Disciplina: Inteligência Artificial e Machine Learning
Trabalho Final de Semestre

~10 minutos de apresentação

O que é o ChefBot?

-
-
-
-

Exemplo de interação:

Usuário → "receitas com frango"

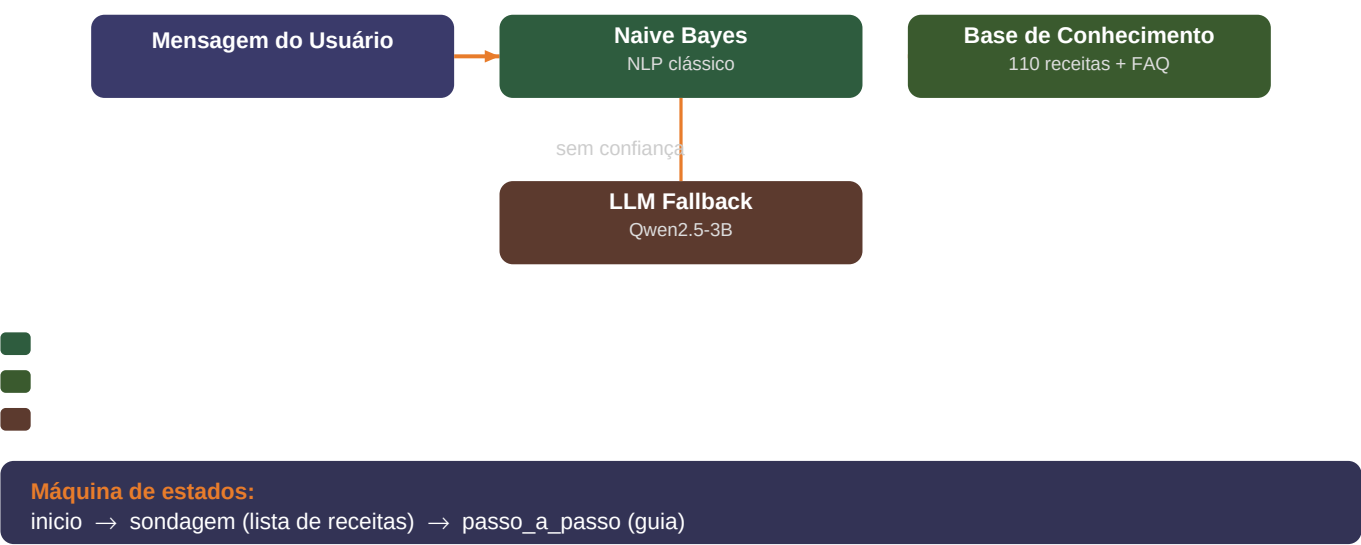
ChefBot → Lista 3 receitas encontradas

Usuário → "a primeira"

ChefBot → Detalhes + "quer guia passo a passo?"

Usuário → "sim" → ChefBot guia cada etapa até o prato ficar pronto

Como o sistema decide a resposta?



O que o ChefBot sabe fazer?

- "receita de feijoada"
- "tenho frango e batata"
- "quero uma sobremesa"
- "me surpreenda"
- guia cada etapa até o fim
- "como amaciar carne?" → FAQ ou LLM
- recusa perguntas fora da culinária
- mostra % base vs LLM em tempo real

Tipos de fonte de resposta rastreados:

- base
- base-faq
- llm
- fora-escopo

Classificação de Intenções (nlp.py)

O texto do usuário é normalizado, tokenizado e classificado com Naive Bayes.

```
# 1. Normaliza: remove acentos, converte para minúsculas
def normalizar(texto):
    return unicodedata.normalize("NFKD", texto)
        .encode("ascii", "ignore").decode().lower()

# 2. Tokeniza: remove pontuação, stopwords e lematiza
def tokenizar(texto):
    tokens = word_tokenize(normalizar(texto), language='portuguese')
    tokens = [t for t in tokens if t.isalpha()]
    return [lemmatizer.lemmatize(t) for t in tokens if t not in stops]

# 3. Classificador Naive Bayes - 171 exemplos, 15 intenções
Se confiança < threshold 1,8x, o LLM reclassifica a intenção como fallback.
def detectar_intencao(texto, fator=1.8):
    .
    .
    .
    return clf.classes_[idx], confiante
```

Busca FAQ (TF-IDF) e Fallback LLM

Perguntas técnicas de culinária vão ao FAQ; o LLM responde o que o FAQ não cobre.

```
# FAQ: 24 pares pergunta-resposta indexados com TF-IDF bigramas
vectorizer_faq = TfidfVectorizer(ngram_range=(1, 2))
faq_matriz = vectorizer_faq.fit_transform(perguntas_faq)

def buscar_faq(pergunta):
    vec = vectorizer_faq.transform([pergunta])
    sims = cosine_similarity(vec, faq_matriz)[0]
    return sims.max()

def classificar(msg):
    # Papel 1: reclassifica intenção
    return inferir(f'Classifique: {INTENCOES}\n{msg}', max_tokens=20)

def dentro_do_escopo(msg):
    # Papel 2: guarda de escopo
    resp = inferir(f'É culinária? SIM ou NAO:\n{msg}', max_tokens=5)
    return 'NAO' not in resp.upper() # dúvida = permitir

def responder(pergunta, historico):
    # Papel 3: resposta aberta
    return pipeline(msgs, max_new_tokens=400, temperature=0.3)
```

Dados e Interface

Base de Receitas (receitas.py)

```
{
  "nome": "Brigadeiro",
  "categoria": "sobremesa",
  "tempo": "30 minutos",
  "dificuldade": "facil",
  "porcoes": 20,
  "ingredientes": [...],
  "instrucoes": [...]
}
```

API REST (app.py)

```
GET /
    Interface web do chatbot

POST /chat
    { mensagem, chat_id }
    → { resposta, fonte, tempo_ms }

GET /stats
    → { pct_base, pct_llm }
```

- Prato Principal** → Feijoada, Moqueca, Estrogonofe
- Sobremesa** → Brigadeiro, Pudim, Bolo de Cenoura
- Lanche** → Coxinha, Pão de Queijo, Tapioca
- Bebida** → Caipirinha, Limonada, Vitamina
- Acompanhamento** → Arroz, Feijão, Farofa, Purê

Como Executar

```
# Instalar dependências
pip install -r requirements.txt

# Iniciar o servidor
python app.py → http://127.0.0.1:5000

# Testar sem LLM (rápido)
```

Conclusão

Observação: baixa modelo Qwen2.5-3B (~5 GB) automaticamente

Objetivo:

NLP Clássico:

LLM:

Resultado:

Aprendizado:

Repositório: github.com/cozta1/chatbotreceitas

Obrigado!